

Dynamic Routes とは

Next.js の App Router では、フォルダ名を `[パラメータ名]` (Slug) にすると、その部分が可変の URL セグメントになります。この手法を **Dynamic Routes** (動的ルーティング) といいます。

API

パス	内容
<code>/shop</code>	店舗一覧
<code>/shop/[id]</code>	店舗詳細 (例: <code>/shop/1</code>)

ShopCard からのリンク

`ShopCard` コンポーネントからは、すでに `/shop/${shop.id}` のリンクがされており、店舗詳細ページではパラメータの `id` を取得して処理します。



渋谷

atelier toi

春の質感は、輪郭の余白から整える。

似合わせカット

透明感カラー

ヘッドスパ

柔らかな光が入る路面店。骨格に合わせたカットと、赤みを抑えたカラー提案が強みです。

評価

4.8 (128件)

詳細を見る



代官山

nook by linen

素の雰囲気、少しだけ意志を足す。

パーマ

ショート

メンズ歓迎

ミニマルな空間で、ショートとニュアンスパーマを中心に提案するプライベートサロンです。

評価

4.7 (89件)

詳細を見る

`<Link href={` /shop/${shop.id} `}>詳細を見る</Link>`

リンク例

`/shop/1`

`/shop/2`

`...`

店舗詳細 API

Route Handler

API 側にも `Dynamic Routes` と同じ仕組みが使えます。 `app/api/shops/[id]/route.ts` を新規作成します。

```
import { NextResponse } from "next/server"
import { MOCK_SHOPS } from "@/data/MockData"

type Params = { params: Promise<{ id: string }> }

export async function GET(_request: Request, { params }: Params) {
  const { id } = await params
  const shop = MOCK_SHOPS.find((s) => s.id === id)

  if (!shop) {
    return NextResponse.json({ error: "not found" }, { status: 404 })
  }

  return NextResponse.json({ shop })
}
```

`Route Handler` の中でも `params` は `Promise` として渡され、 `page.tsx` と同じように `await` して取り出します。

ShopService

`ShopService` (`lib/services/shops.ts`) から `API` を呼び出す `getShop(id)` を追加します。

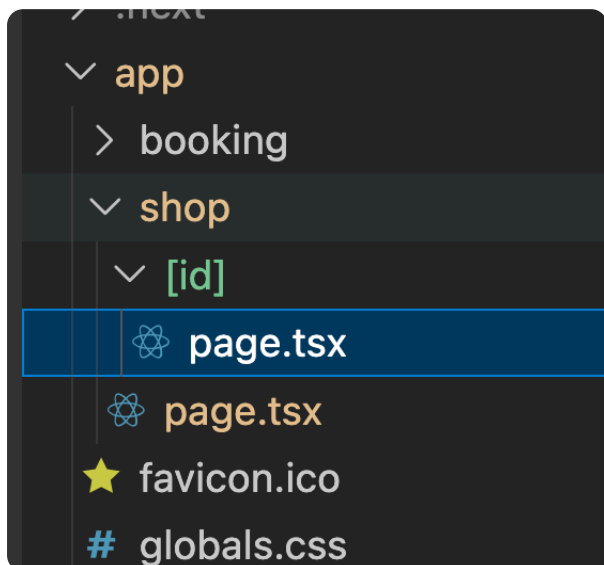
```
export async function getShop(id: string): Promise<Shop | null> {
  const res = await fetch(`${BASE_URL}/api/shops/${id}`, { cache: "no-store" })
  if (res.status === 404) return null

  const data: { shop: Shop } = await res.json()
  return data.shop
}
```

分岐	意味
<code>res.status === 404</code>	Route Handler が「見つからない」と返してきた場合、null を返す

店舗詳細ページ

`app/shop/[id]/page.tsx` を新規作成します。 **Dynamic Routes** ではフォルダ名を `[id]` や `[slug]` とするのがポイントです。



params 定義

動的ルートの `params` は **Promise** として渡されます。店舗詳細ページでは店舗ID `id` を `string` 型の Props で定義します。

```
// Props のデータ型
type Props = {
  params: Promise<{ id: string }>
}

// コンポーネントの params
export default async function ShopDetailPage({ params }: Props) {
}
```

id 取得（非同期）

Server Component では非同期 `await` で `id` を取り出し、 `ShopService` の `getShop(id)` に渡します。

```
import { getShop } from "@lib/services/shops"

export default async function ShopDetailPage({ params }: Props) {
  // 非同期で id を取得
  const { id } = await params
  // ShopService 経由で1件取得
  const shop = await getShop(id)

  return (
    <div>
      <h1 className="text-2xl font-bold">{shop?.name}</h1>
    </div>
  )
}
```

6章で作った `ShopService` をここでも再利用します。ページ側は `Route Handler` や `fetch` の詳細を意識せず、`getShop(id)` を呼ぶだけで済みます。

Next Salon

店舗

予約

マイページ

atelier toi

?. (オプショナルチェーン)

`?.` は オプショナルチェーン (Optional Chaining) と呼ばれるJavaScript / TypeScript の書き方です。

```
<h1 className="text-2xl font-bold">{shop?.name}</h1>
```

- shop にデータがあれば name を取得する。
- shop が null や undefined ならエラーにせず undefined にする

404 処理

存在しない `id` がアクセスされた場合（例: `/shop/999`）、`notFound()` を使って安全に 404 ページを返します。

```
import { notFound } from "next/navigation"
import { getShop } from "@/lib/services/shops"

type Props = {
  params: Promise<{ id: string }>
}

export default async function ShopDetailPage({ params }: Props) {
  const { id } = await params
  const shop = await getShop(id)

  // Notfound 404
  if (!shop) {
    notFound()
  }

  return (
    <div>
      <h1 className="text-2xl font-bold">{shop.name}</h1>
    </div>
  )
}
```

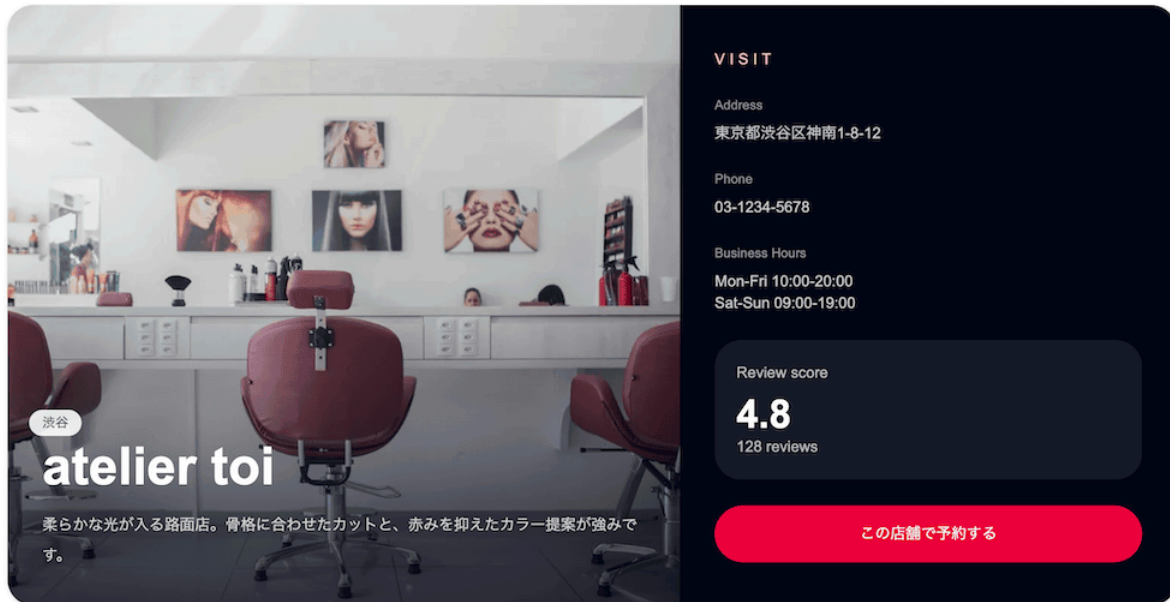
404

This page could not be found.

`notFound()` を呼ぶと、その場で処理が中断され `app/not-found.tsx`（未作成の場合は Next.js 標準の 404 UI）が表示されます。

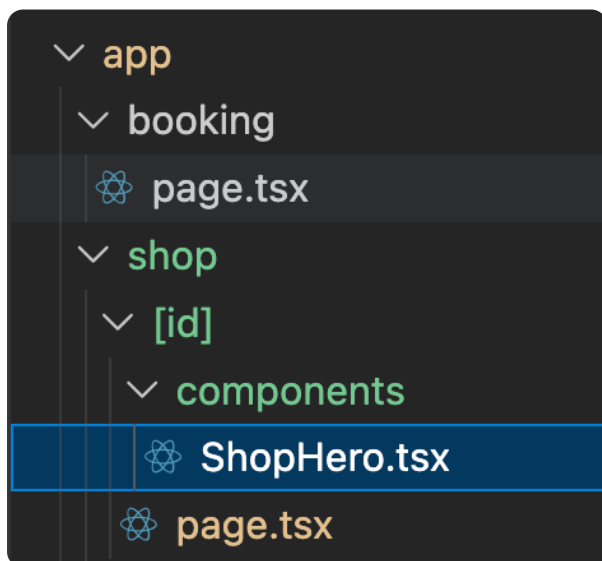
ShopHero コンポーネント

ヒーロー部分は JSX が長くなるため、`page.tsx` に直接書かず、専用コンポーネントに分けます。



コンポーネント作成

`components/shop/ShopHero.tsx` を作成します。



グリッドレイアウト

ヒーローエリアは「画像 + 情報パネル」の 2 カラム構成にします。

```
<div className="grid lg:grid-cols-[1.15fr_0.85fr]">
  <div>{/* 画像エリア */}</div>
  <div>{/* 情報パネル */}</div>
</div>
```

grid-cols

Tailwind の `grid-cols-[...]` には任意の比率を指定できます。`fr` は fraction（割合・分数）の略で、**利用できるスペースを何分ずつ使うか**を表す単位です。

$1.15fr + 0.85fr = 2fr$ でグリッドレイアウトした場合です。

```
lg:grid-cols-[1.15fr_0.85fr]
```

項目	指定	割合
左	1.15fr	57.5%
右	0.85fr	42.5%

店舗詳細

左側には店舗名や画像を表示、右カラムには、店舗詳細や予約リンク（`/shop/[id]/book`）などを配置します。

```

import Image from "next/image"
import Link from "next/link"

import type { Shop } from "@types/shop"

export default function ShopHero({ shop }: { shop: Shop }) {
  return (
    <section className="overflow-hidden rounded-3xl bg-white shadow-sm">
      <div className="grid lg:grid-cols-[1.15fr_0.85fr]">
        <div className="relative min-h-96">
          <Image
            src={shop.coverImage}
            alt={shop.name}
            fill
            priority
            className="object-cover"
            sizes="(max-width: 1024px) 100vw, 60vw"
          />
          <div className="absolute inset-0 bg-linear-to-t from-slate-950/75 via-slate-950/10 to-transparent" />
          <div className="absolute inset-x-0 bottom-0 p-8 text-white">
            <div className="absolute left-5 top-5 rounded-full bg-white/90 px-3 py-1 text-xs font-medium text-slate-700">
              {shop.area}
            </div>
            <h1 className="mt-4 text-4xl font-semibold sm:text-5xl">
              {shop.name}
            </h1>
            <p className="mt-4 max-w-2xl text-sm leading-7 text-white/80">
              {shop.description}
            </p>
          </div>
        </div>

        <div className="bg-slate-950 px-8 py-10 text-white">
          <p className="text-sm font-medium tracking-[0.3em] text-rose-200 uppercase">
            Visit
          </p>
          <div className="mt-6 space-y-6">
            <div>
              <p className="text-xs text-white/55">Tags</p>
              <div className="mt-2 flex flex-wrap gap-2">
                {shop.tags.map((tag) => (
                  <span
                    key={tag}
                    className="rounded-full bg-white/10 px-3 py-1 text-xs font-medium text-white/85"
                  >
                    {tag}
                  </span>
                ))}
              </div>
            </div>
          </div>
        </div>
      </div>
    </section>
  )
}

```

```

<div className="rounded-3xl bg-white/8 p-5">
  <p className="text-sm text-white/70">Review score</p>
  <p className="mt-2 text-4xl font-semibold">{shop.rating}</p>
  <p className="text-sm text-white/70">
    {shop.reviewCount} reviews
  </p>
</div>
<Link
  href={`\shop/${shop.id}/book`}
  className="inline-flex w-full items-center justify-center rounded-full bg-rose-600 px-5 py-
4 text-sm font-medium text-white transition hover:bg-rose-500"
>
  この店舗で予約する
</Link>
</div>
</div>
</section>
)
}

```

ポイント: `ShopHero` はクリックイベントや `state` を使わないため、`"use client"` は不要です。Server Component のまま `next/image` や `next/link` を使えます。

最後に `app/shop/[id]/page.tsx` から `ShopHero` を呼び出します。

```
import { notFound } from "next/navigation"

import { getShop } from "@lib/services/shops"
import ShopHero from "@components/shop/ShopHero"

type Props = {
  params: Promise<{ id: string }>
}

export default async function ShopDetailPage({ params }: Props) {
  const { id } = await params

  const shop = await getShop(id)
  if (!shop) {
    notFound()
  }

  return (
    <div>
      <ShopHero shop={shop} />
    </div>
  )
}
```

動作確認

- `/shop/1` などの正しい ID → 店舗詳細（ヒーロー部分）が表示される

Next Salon 店舗 予約



代官山

Atelier

落ち着いた空間で、丁寧なカウンセリングを大切にしています。

VISIT

Tags

カット カラー 髪質改善

Review score

4.9

42 reviews

この店舗で予約する

- `/shop/999` などの存在しない ID → 404 ページが表示される

Next Salon 店舗 予約

404

This page could not be found.

N 1 Issue X

まとめ

テーマ	ポイント
動的ルーティング	<code>[id]</code> フォルダで可変 URL セグメントを表現
<code>params</code>	Next.js 15 では <code>Promise</code> として渡される。 <code>await</code> して取り出す
<code>notFound()</code>	該当データがない場合に 404 を返す。以降 TypeScript の型も絞り込まれる
<code>generateMetadata</code>	ページごとに動的にタイトル・説明文を生成できる
Route Handler の動的セグメント	page.tsx と同じ <code>[id]</code> の仕組みが app/api/ 配下でも使える
ShopService の再利用	一覧・詳細どちらも <code>getShop</code> / <code>getShops</code> を呼ぶだけで、Route Handler の詳細をページ側が意識しない
<code>grid-cols- [...]</code>	Tailwind で任意の比率の 2 カラムレイアウトを組める
コンポーネントの Props 化	<code>SectionHeading</code> にデフォルト引数付き Props を持たせ、複数ページで再利用できるようにする